EXPERIMENT NO. 8

Post-Lab Questions and Answers

Name: Hitanshu Shah

Roll Number / SAP ID: S069/60018240010

1. Explain the difference between LLMOps and AgentOps.

LLMOps (Large Language Model Operations) focuses on the development, deployment, monitoring, and maintenance of applications that use large language models. It includes managing prompts and model versions, evaluating responses, monitoring latency and cost, and ensuring reliability. AgentOps focuses on the complete lifecycle of AI agents that can plan tasks, choose and use tools, perform multiple steps, and respond to the results of their actions. It places additional emphasis on tool selection, execution traces, intermediate steps, workflow coordination, and whether the agent completes the task successfully. For example, LLMOps may monitor the quality and response time of a language model, while AgentOps can track whether an agent selected the correct calculator tool and used its result. Thus, LLMOps manages LLM-powered applications, whereas AgentOps focuses on the operation and evaluation of AI agents and their workflows.

2. What is LangSmith, and why is it used?

LangSmith is an observability, debugging, and evaluation platform for applications built with language models and AI agents. It records execution traces that show the steps performed during a request, including model calls, tool invocations, intermediate outputs, and errors. These traces help developers understand why an agent selected a particular tool, identify incorrect arguments or unnecessary calls, and locate failures. LangSmith can also help measure performance, inspect latency and token usage when available, evaluate tasks against test cases, and compare different prompts or agent configurations. In this experiment, it is used to monitor the LangChain agent while it answers ten test queries. By examining the traces for Prompt V1 and Prompt V2, we can compare tool-calling accuracy, task success, execution time, and token consumption, then identify changes that may improve the agent's reliability and efficiency.

3. How does prompt modification affect agent performance?

Prompt modification changes the instructions that guide an AI agent's behavior. A clear system prompt can specify when to call a tool, which tool to select for a particular request, how to handle missing or invalid inputs, and when to answer without using a tool. These rules can improve tool-selection accuracy, task success, consistency, and error handling. For example, Prompt V2 in this experiment explicitly instructs the agent to use the calculator for arithmetic and the student information tool for student queries. It also tells the agent not to invent records or make unnecessary calls. However, a more detailed prompt can increase the amount of text processed and may raise token consumption or latency. Therefore, a prompt should not be judged only by whether its answers sound correct. Both prompt versions should be tested on the same benchmark queries and compared using task success rate, tool-calling accuracy, average latency, and token consumption. The best prompt is the one that provides reliable results with an appropriate balance of accuracy and efficiency.